

Interview Q&A: numpy

1. What is the difference between numpy's `ndarray` and `masked\_array`?

A masked array is a subclass of ndarray that allows for missing or invalid data to be represented using masks.

2. How does NumPy's ufunc handle NaN values?

NumPy's ufunc will propagate NaN values through the operation, unless explicitly handled with a function like `numpy.isnan()` or `numpy.isfinite()`. This is because NaN represents an unknown or unreliable value.

3. What is the purpose of `numpy.ufunc.reduce`?

It applies an ufunc to all elements in a given array, reducing it to a single value.

4. How does NumPy's `broadcasting` work when dealing with arrays of different shapes?

NumPy's broadcasting works by aligning the dimensions of the input arrays, allowing operations to be performed element-wise even if the arrays have different shapes.

5. What is the difference between numpy's `isclose` function and `allclose`?

The `isclose` function checks if two arrays are element-wise equal within a tolerance, while `allclose` checks if two arrays are element-wise equal within a tolerance for all elements.

6. What is the purpose of numpy's `einsum` function?

The `einsum` function provides a compact way to perform multi-dimensional array operations using the Einstein summation convention.

7. What is the purpose of `numpy.einsum` when used with a string expression?

It allows for efficient computation of large arrays by specifying the operations to be performed on specific axes.

8. How does NumPy's `numpy.vectorize` function work, and what are its limitations?

NumPy's `vectorize` function converts a Python function into an element-wise vectorized ufunc. However, it can be slower than using native NumPy functions due to the overhead of converting between Python objects and NumPy arrays.

9. What is the difference between numpy's `nan\_to\_num` function and `numpy.isnan`?

The ``nan_to_num`` function replaces NaN values with a specified value, while ``numpy.isnan`` checks if a value is NaN.

10. How does NumPy's ``np.dot()`` function handle matrix multiplication when the input arrays have more than two dimensions?

NumPy's ``np.dot()`` function will perform a batched dot product, where it treats the last two axes of each array as the axes to multiply.

11. What is the purpose of ``numpy.ufunc.at``?

The ``at`` attribute of a ufunc allows it to operate on a single element of an array, rather than the entire array.

12. What is the difference between numpy's ``numpy.ufunc.at`` and ``numpy.ufunc.reduce``?

`ufunc.at` applies an operation to a single element of an array, while `ufunc.reduce` accumulates values from an array into a scalar value.

13. What is the purpose of ``numpy.ufunc.at`` when used with an array?

It applies a ufunc to a single element of an array, specified by its index.

14. How does NumPy's ``numpy.ufunc.at`` function handle out-of-bounds indexing?

NumPy's ``numpy.ufunc.at`` function uses the same indexing rules as regular array assignment, so it will raise an error if the index is out of bounds.

15. What is the difference between numpy's ``numpy.ufunc.at`` and ``numpy.ufunc.reduce`` when used with an array?

`ufunc.at` applies the operation to a single element of the array, while `ufunc.reduce` accumulates the result across all elements.

16. How does NumPy's ``numpy.ufunc.at`` function handle out-of-bounds indexing when used with an array?

NumPy's ``numpy.ufunc.at`` function raises a ``IndexError`` when attempting to access an out-of-bounds index.